How to Execute : Mini CPU Simulator in emu8086

This guide covers:

- **Part A**: running the project in emu8086
- **Part B**: explaining the code to sir inside emu8086
- **Part C**: running it and explaining the output
- **Part D**: viva questions

For a **line-by-line explanation of the code** (how we made it, and what each line does), see **CODE_EXPLANATION.md (also a PDF in this folder)**.

Repository: https://github.com/lakshay8toic/Mini-CPU-Simulator

---

Part A : Running the project in emu8086

Step 1 : Get the file onto the laptop that has emu8086

You only need **one file**: `MINICPU.ASM`. Use any of these ways:

- **Download from GitHub (easiest):** on that laptop, open **https://github.com/lakshay8toic/Mini-CPU-Simulator**, click **src**, then **MINICPU.ASM**, then the **download button** ⬇. The file goes to the Downloads folder.
- **Whole project as a ZIP:** click the green **Code** button, then **Download ZIP**, then **Extract All**. The file is in `src\MINICPU.ASM`.
- **Pen drive or WhatsApp:** copy `MINICPU.ASM` from your phone or laptop to the Desktop.

Do this at least one day before the demo, and do one full run on that laptop (Steps 2 and 3).

Step 2 : Open the code

1. Start **emu8086**. Close the welcome window if one appears.
2. Click **Open** and select `MINICPU.ASM` (from Downloads, Desktop, or the extracted `src` folder).
3. The full code appears in the editor. **This is where you explain the code (Part B).**

Step 3 : Compile and run

1. Click **Emulate** on the toolbar (or press **F5**).
2. If emu8086 asks to save `MINICPU.com`, click **Save**.
3. In the emulator window, drag the **step delay** slider all the way left (**0 ms**) so the program runs fast.

4. Click **Run**.
5. The black **emulator screen** window opens. Click once inside it, then press any key for each step.

Problem	Fix
Keys do nothing	Click inside the black emulator screen window first
The screen window is missing	Click the **screen** button in the emulator window
You want to run it again	Click **Reload**, then **Run**

Part B : Explaining the code (in the emu8086 editor)

The code has 5 parts. Scroll through them in this order.

1. The header comment (top of the file)

> "This program simulates a small CPU with registers PC, MAR, MDR, IR and ACC. Each instruction is 1 byte: the high 4 bits are the opcode and the low 4 bits are the address. It has 6 instructions: LOAD, ADD, SUB, STORE, JMP and HALT."

2. The memory : `ram` (around line 50)

```
ram     db 1Dh, 2Eh, 3Fh, 4Ch, 56h, 2Eh, 00h, 00h
        db 00h, 00h, 00h, 00h, 00h, 09h, 05h, 03h
```

> "This is the memory of the small CPU, 16 bytes. The first bytes are the program, and the last three bytes are the data: 9, 5 and 3. For example, `1D` means opcode 1 (LOAD) and address D, so it loads the value at address 0D into ACC."

3. The registers : `cpu_pc`, `cpu_mar`, `cpu_mdr`, `cpu_ir`, `cpu_acc` (around line 56)

> "Each register of the small CPU is one byte variable in the 8086 program."

4. The main loop : `cpu_cycle` (around line 109)

```
cpu_cycle:
    call wait_key
    call fetch          ; step 1
    call wait_key
    call decode         ; step 2
    call wait_key
    call execute        ; step 3
    ...
    jne cpu_cycle       ; repeat until HALT
```

> "This is the instruction cycle. It waits for a key, then runs fetch, decode and execute, and repeats until the CPU executes HALT."

5. The three steps

`fetch` (around line 145)

> "MAR gets the PC, the byte at that address is read into MDR, MDR is copied into IR, and PC is increased by 1."

`decode` (around line 169)

```
 shr al, cl      ; shift right 4 times -> opcode
and al, 0Fh     ; keep low 4 bits    -> address
```

> "Shifting IR right by 4 bits gives the opcode. ANDing IR with 0F keeps the low 4 bits, which is the address."

`execute` (around line 194)

> "We compare the opcode with 1, 2, 3, 4 and 5 and jump to the matching code. For example, `do_add` reads the memory value and adds it to ACC with the 8086's own ADD instruction."

The helper routines at the bottom (`print_str`, `print_hex`, `wait_key`) only print text and read keys, using **INT 21h**.

Part C : Running it and explaining the output

After clicking **Run**, the screen shows the memory:

```
 Address : 00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F
Memory  : 1D 2E 3F 4C 56 2E 00 00 00 00 00 00 00 09 05 03
```

Press a key **three times for each instruction** (Fetch, Decode, Execute). For the first instruction:

Key	Screen shows	What to say
1st	`[FETCH] ... PC=01 MAR=00 MDR=1D IR=1D`	"The instruction 1D is fetched from address 00, and PC becomes 01."
2nd	`[DECODE] opcode = 01, address = 0D`	"Opcode 1 is LOAD, and the address is 0D."
3rd	`[EXECUTE] LOAD ... ACC=09`	"The value 9 from address 0D is loaded into ACC."

Keep pressing keys and point out these moments:

Instruction	What to point at
ADD 0E	ACC = **0E** (9 + 5 = 14)
SUB 0F	ACC = **0B** (14 − 3 = 11)
STORE 0C	MAR = 0C, MDR = 0B. The result is written to memory
JMP 06	PC = **06**, so address 05 is skipped
HALT	The CPU stops

The final screen shows:

```
========== CPU HALTED ==========
Result at address 0C = 0B (hex)
Memory  : 1D 2E 3F 4C 56 2E 00 00 00 00 00 00 0B 09 05 03
```

> "0B in hex is 11, so 9 + 5 − 3 = 11, and you can see it saved at address 0C in memory."

Optional : change the numbers live

If sir asks you to try other numbers, edit the last three values in the `ram` line. For example, change `09h, 05h, 03h` to `14h, 06h, 02h` (20 + 6 − 2). Click **Emulate** and **Run** again. The result is **18** hex = 24.

Part D : Viva questions

Q1. What does this project do? It simulates a small CPU in 8086 assembly and shows the fetch, decode and execute steps of every instruction, including how PC, MAR, MDR, IR and ACC change.

Q2. What happens in the fetch step? MAR ← PC, MDR ← M[MAR], IR ← MDR, PC ← PC + 1.

Q3. How is the instruction decoded? The instruction is 1 byte. Shifting it right 4 bits (`SHR`) gives the opcode. ANDing it with `0Fh` gives the address.

Q4. What is the job of MAR and MDR? MAR holds the memory address being accessed. MDR holds the data read from, or written to, that address.

Q5. Why does PC increase by 1? Each instruction is 1 byte, so the next instruction is at the next address.

Q6. How does JMP work? JMP puts the address part of the instruction into PC, so the next fetch happens from that address. In this program it skips address 05.

Q7. What is the accumulator? ACC is the register that holds the result of LOAD, ADD and SUB. STORE copies it into memory.

Q8. Which 8086 instructions and interrupts did you use?

- `MOV`, `ADD`, `SUB`, `CMP`, `JE`, `JMP`, `SHR`, `AND`, `CALL` and `RET`
- **INT 21h** function 09h to print a message, 02h to print a character, 07h to read a key, and 4Ch to exit